Homework #2

2.3T4 2.4T4 2.5T8 3.1T12 3.2T6 3.3T9

2.3T4

4. 考虑下面的算法。

算法 Mystery(n)
//输入：非负整数 n
$S \leftarrow 0$
for $i \leftarrow 1$ **to** n **do**
$S \leftarrow S + i * i$
return S

a. 该算法求的是什么？
b. 它的基本操作是什么？
c. 该基本操作执行了多少次？
d. 该算法的效率类型是什么？
e. 对该算法进行改进，或者设计一个更好的算法，然后指出它们的效率类型。如果做不到这一点，请试着证明这是不可能做到的。

解. a. $S=\sum_{i=1}^{n} i^2$，即从1到n的平方和

b. 乘法

c. $C(n)=n$

d. $C(n)=n \in \Theta(n)$，线性

e. 使用 $S=\sum_{i=1}^{n} i^2=\frac{n(n+1)(2n+1)}{6}$ 直接计算

此时 $C(n) \in \Theta(1)$，为常量

2.4T4

4. 考虑下面的递归算法。

算法 $Q(n)$
//输入：正整数 n
if $n = 1$ **return** 1
else return $Q(n-1)+2*n-1$

a. 建立该函数值的递推关系并求解，以确定该算法计算的是什么。
b. 建立该算法所做的乘法运算次数的递推关系并求解。
c. 建立该算法所做的加减运算次数的递推关系并求解。

解. a. $Q(n)=\begin{cases} Q(n-1)+2n-1 & n>1 \\ 1 & n=1 \end{cases}$

$\Rightarrow Q(n)=\sum_{i=1}^{n}(2n-1)=n^2$，该算法计算 $Q(n)=n^2$

b. $M(n)=\begin{cases} M(n-1)+1 & n>1 \\ 0 & n=1 \end{cases}$

$\Rightarrow M(n)=n-1 \in \Theta(n)$

c. $C(n)=\begin{cases} C(n-1)+2 & n>1 \\ 0 & n=1 \end{cases}$

若考虑Q(n-1)中的减法

则 $C(n)=3n-3 \in \Theta(n)$

$\Rightarrow C(n)=2n-2 \in \Theta(n)$

2.5T8

8. 改进算法Fib，使它只需要$\Theta(1)$的额外空间。

```
AdvancedFib(n)
// 计算第n个Fib数
// 输入：非负整数n
// 输出：第n个Fib数
a←0; b←1
for i←2 to n do
    b←a+b
    a←b-a
if n=0 return 0
else return b
```

3.1 T12

12. **a.** 请证明，如果冒泡排序对列表比较一遍之后没有交换元素的位置，那么这个表已经排好序了，算法也可以停止了。

b. 结合所做的改进，为冒泡排序写一段伪代码。

c. 请证明改进版本的最差效率也是平方级的。

a. 证明：由于没有交换位置，

故 $A_0 \le A_1 \le A_2 \cdots \le A_{n-1}$，已排比序，可停止

b.

```
BubbleSort(A[0,...n-1])
// 对A[0...n-1]进行排序
// 输入：可排序数组A[0..n-1]
// 输出：非降序数组A[0..n-1]
for i←0 to n-2 do
    flag=True
    for j←0 to n-2-i do
        if A[j+1]<A[j]
            swap A[j] and A[j+1]
            flag=false
    if flag
        return
```

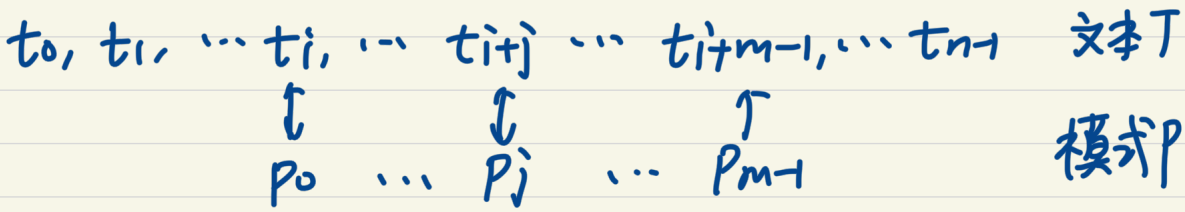
c. 最差效率仍为倒序排列，不会提前停止

$$C(n)=\sum_{i=0}^{n-2}\sum_{j=0}^{n-2-i}1=\sum_{i=0}^{n-2}(n-1-i)=\frac{n(n-1)}{2}\in\Theta(n^2)$$

3.2 T6

6. 给出一个长度为 n 的文本和长度为 m 的模式构成的实例，它是蛮力字符串匹配算法的一个最差输入。请确切指出，对于这样的输入需要做多少次字符比较运算。

解：



需要 $(n-m+1)m$ 次

3.3 T9

9. 对于一个平面上 $n>1$ 个点的集合，设计一个线性效率的算法来求出其凸包的两个极点。

解：

```
FindEPoints( P[0..n-1])
//找到两极点
//输入：n点数组P[0..n-1],每个点都有x,y
//输出两极点,
min_point ← P[0]
max_point ← p[0]
for i←1 to n-1 do
    if P[i].x < min_point.x or (P[i]==min_point.x
    and P[i].y < min_point.y)
        min_point ← P[i]
if P[i].x > max_point.x or (P[i]==max_point.x
```

```
    and P[i].y > max_point.y)
        max_point ← P[i]
return (min_point, max_point)
```